Forging Dilithium and Falcon Signatures by Single Fault Injection

Sven Bauer, Fabrizio De Santis
Siemens AG, Foundational Technology
Munich, Germany
{svenbauer, fabrizio.desantis}@siemens.com

December 4, 2024

Abstract

Embedded devices commonly rely on digital signatures to ensure both integrity and authentication. For example, digital signatures are typically verified during the boot process or firmware updates to verify the integrity of a system. They are also used to ensure authenticity of a communication party in secure protocols.

Fault injection can be used to tamper with a device in order to cause malfunctioning during cryptographic computations. For example, fault injections can be used to disturb digital signing operations. With the right type of fault an attacker can compute private keys from faulted signatures. However, fault injections can also be used during verification to get maliciously crafted digital signatures accepted during signature verification with catastrophic consequences for the security of an embedded device.

In this paper, we introduce new non-obvious fault injection attacks on the verification routines of Dilithium and Falcon signature schemes, which allow an attacker to get signatures for arbitrary messages accepted by fault injection. We demonstrate the feasibility of our attacks by simulations using an ARM Cortex-M4 and the `pqm4` library as a target of evaluation and pinpoint vulnerable instructions. Finally, we propose and discuss possible countermeasures against these attacks.

1 Introduction

Embedded devices rely heavily on the verification of digital signatures to ensure the authenticity and integrity of data. During the boot process, the device verifies the digital signature of the firmware to ensure that it has not been tampered with or modified. Similarly, software updates are verified using digital signatures to ensure that only authorized and unaltered software is installed.

Moreover, digital signatures are also used to verify the authenticity of a communication party. When two devices communicate with each other, signatures are often a part of an authentication protocol or digital certificates. These signatures are then verified to confirm that both parties are authentic and not being impersonated.

Hence, if an attacker manages to bypass or compromise the signature verification process, they can inject malicious code into the device or impersonate a legitimate communication party. This can lead to severe consequences, including data breaches, financial losses, and reputation damage. Therefore, it is crucial to ensure that signature verification is implemented securely and robustly in embedded devices.

In [13] Muir showed how to trick a device into accepting a maliciously crafted RSA signature in a non-obvious way by fault injection. This attack is particularly noteworthy because it does not focus on the most vulnerable part of RSA signature verification, which is the comparison between the computed signature and the provided signature. Typically, an attacker would aim at exploiting this comparison through fault attacks. However, skilled developers are aware of this vulnerability and take measures to mitigate it. Muir's attack showed that there might be non-obvious ways to bypass existing countermeasures by fault injection.

In July 2022, the U.S. Department of Commerce's National Institute for Standard and Technology (NIST) selected three digital signature algorithms Dilithium [9], Falcon [15] and Sphincs+ [7] as quantum-computer resistant replacements for RSA and elliptic-curve based digital signatures [21]. Dilithium and Sphincs+ were standardized (with small modifications) in August 2024 as ML-DSA [20] and SLH-DSA 205 [22], respectively. We will continue to use the name Dilithium rather than ML-DSA because this is the name used in most of the literature and source code referred to in this paper and because the small differences with ML-DSA are not relevant in the present context. The standardization of Falcon is still ongoing as of December 2024. Falcon will be standardized under the name FN-DSA, but we continue to use the name Falcon for the same reasons as for using the name Dilithium rather than ML-DSA. Dilithium and Falcon are good candidates for general purpose applications because they offer smaller signatures than Sphincs+.

Muir's attack motivates considering the possibility of fault injection attacks against post-quantum signature schemes. As for RSA verification, there are obvious places for a fault injection attack in Dilithium and Falcon as well, e.g., the final decision whether to accept or reject a signature, or the final length-check in the Falcon signature verification procedure (see Algorithm 2, Algorithm 2). However, as in the case of RSA, there may be less obvious attacks for Dilithium and Falcon verification routines that developers are more likely to overlook. Consequently, these vulnerabilities may not be adequately protected by appropriate countermeasures in practice. This is especially important for post-quantum signature schemes, as they are relatively new and their implementations may not have been subjected to the same level of scrutiny yet, as more established schemes. By identifying and testing these attack vectors,

we can help to improve the security of implementations of these schemes and to prevent potential attacks in the future.

In this paper, we describe fault attacks against parts of the DILITHIUM and FALCON signature verification procedure which are not such obvious targets as the final comparison. We show that these attacks are successful on the assumption that a fault injection attack can cause the target hardware to skip instructions.

A good overview of instruction skipping by fault injection can be found in [12], which also shows that this is a very realistic fault model. See also [5] and [11] for recent detailed analyses of the instruction skip fault model.

1.1 Previous work

Muir's attack [13] is a generalization of an attack by Seifert described in [18]. It tricks a device into accepting an incorrect RSA-signature with a fault injection attack.

Muir's attack works as follows: the attacker flips a few bits in the public RSA modulus n to obtain a number n' that is easy to decompose into prime-factors. Because a randomly chosen integer is likely to have many small prime factors and is hence easy to factor, such an n' can typically be found with only a few bit flips. With the factorization of n' it is easy to compute $\phi(n')$ and then $d' = 1/e \bmod \phi(n')$[1]. Now the attacker simply signs an arbitrary message with (d', n') and sends the signed message and the crafted signature to the device. With a fault injection attack during the signature verification procedure, the attacker tries to reproduce the bit flips to turn n into n' in the device. If this succeeds, the device accepts the crafted signature as valid.

In the field of post-quantum cryptography, a number of publications have already addressed fault attacks against post-quantum signature schemes. Prominent examples are [3, 10, 1]. However, these works do not target the signature verification procedure. A number of fault attacks against BLISS, ring-TESLA, and the GLP-scheme, including their verification procedure, can be found in [2]. Fault attacks against signature verification in DILITHIUM are briefly discussed in [16, 17]. However, these publications describe generic, more obvious, attacks, namely, targeting the final comparison. Additionally, [17] presents an attack based on zeroing the twiddle factors during the computation of the Number-Theoretic Transform (NTT).

1.2 Contributions

This paper shows a number of non-obvious, yet practically simple and realistic fault injection attacks against the verification routines of the post-quantum signature schemes DILITHIUM and FALCON. The attacks target parts of the code which even an experienced developer would not necessarily consider protecting

[1] In the unlikely case that $\gcd(n', e) \neq 1$, the attacker simply tries again with a different n'.

against fault injections. Consequently, an attacker can trick a device into processing arbitrary unauthenticated data, hence potentially enabling the attacker to install malicious software on a device. If signature verification during the establishment of a secure communication session is attacked, the attacker can force the device to communicate with an unauthenticated party.

1.3 Outline

In Section 2, a description of the DILITHIUM signature scheme is provided. In Section 3 we show an attack against DILITHIUM signature verification. Section 4 gives a description of the FALCON signature scheme. In Section 5 we show how to attack FALCON signature verification. A practical evaluation of the proposed attacks is provided in Section 6. Possible countermeasures to protect against the proposed fault attacks are described in Section 7. We summarize our conclusions and give an outlook in Section 8.

2 Dilithium signature verification

We give a brief description of the DILITHIUM signature verification procedure. For the full details of the DILITHIUM signature scheme, we refer the reader to the DILITHIUM specification [9].

DILITHIUM largely works over a ring $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ with $q = 8380417$. The specification defines a representation of elements of R_q as byte strings. Hence, we will, for example, apply hash functions to elements of R_q, implicitly assuming the encoding defined in the specification[2].

The DILITHIUM specification [9] defines three variants, targeting NIST security levels 2, 3 and 5 (see [23, Section 4.A.5] for a definition of these security levels). Therefore, signature verification is parameterized. We give the values of the parameters for the three security levels in Table 1. The DILITHIUM specification lists more parameters, but we have only listed those which are relevant to signature verification.

A DILITHIUM public key is a pair $(\rho, \mathbf{t}_1) \in \{0,1\}^{256} \times R_q^l$. A DILITHIUM signature is a triple $(\tilde{c}, \mathbf{z}, \mathbf{h}) \in \{0,1\}^{256} \times R_q^l \times \{0,1\}^{256k}$. The signature verification procedure calls a function $\texttt{ExpandA} : \{0,1\}^{256} \to R_q^{k \times l}$ that is used to expand the bit string ρ to a matrix $A \in R_q^{k \times l}$. So ρ is a compressed representation of A. Furthermore, the signature verification procedure calls a subroutine `SampleInBall` that produces a pseudo-random 256-bit string with exactly τ bits equal to one from an input seed. Finally, a function $\texttt{UseHint}_q$ is needed that takes as input a 256-bit string (the "hint"), a vector in R_q^k and an integer (the "low-order rounding range").

[2] The exact definition of this encoding does not matter for the purposes of this paper and can be found in [9]

Table 1: Parameters of DILITHIUM signature verification

NIST security level	2	3	5
γ_1	2^{17}	2^{19}	2^{19}
γ_2	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
(k,l)	$(4,4)$	$(6,5)$	$(8,7)$
β	78	196	120
ω	80	55	75

Algorithm 1 DILITHIUM.VERIFY(m, sig, pk)

Require: A message m a signature $\texttt{sig} = (\tilde{c}, \mathbf{z}, \mathbf{h})$, a public key $\texttt{pk} = (\rho, \mathbf{t}_1)$.
Ensure: Accept or reject

$A \leftarrow \texttt{ExpandA}(\rho)$
$\mu \leftarrow \texttt{SHAKE256}(\texttt{SHAKE256}(\rho \| \mathbf{t}_1, 256) \| m, 512)$
$c \leftarrow \texttt{SampleInBall}(\tilde{c})$
$\mathbf{w}_1' \leftarrow \texttt{UseHint}(\mathbf{h}, \mathbf{Az} - c\mathbf{t}_1, 2\gamma_2)$
if $\|z\|_\infty < \gamma_1 - \beta$ and $\tilde{c} = \texttt{SHAKE256}(\mu \| \mathbf{w}_1', 256)$ and the number of 1s in $\mathbf{h}$ is $\leq \omega$ **then**
    accept
else
    reject

3 Fault attacks against Dilithium signature verification

We use the same notation as in Section 2 and assume the attacker has access to a valid DILITHIUM public key $(\rho, \mathbf{t}_1)$ and some message m with a valid DILITHIUM signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ that can be verified with the public key $(\rho, \mathbf{t}_1)$.

We assume further the attacker has physical access to a device that only accepts messages with signatures that can be verified under the public key $(\rho, \mathbf{t}_1)$.

The attacker wants to force the device into accepting a message m' for which the attacker does not have a valid signature.

Our attack consists of two steps: first, the attacker constructs an invalid signature σ' for m' and then, using a fault injection, forces the device to accept this as a valid signature for m'. The point is, that σ' needs to be specifically crafted to make this fault injection attack practical.

To achieve this, the attacker proceeds as follows:

1. Calculate:

$$\mu' := \texttt{SHAKE256}(\texttt{SHAKE256}(\rho \| \mathbf{t}_1, 256) \| m', 512)) \tag{1}$$

This is the same step as in the verification procedure in Algorithm 1 when verifying a signature for m'.

2. Compute:
$$\mathbf{w}_1'' := \texttt{UseHint}_q(\mathbf{h}, \mathbf{Az}, 2\gamma_2) \tag{2}$$
This is the same step as in the verification procedure algorithm in Algorithm 1 in Algorithm 1, but with $c = 0$.

3. Compute:
$$\tilde{c}' := \texttt{SHAKE256}(\mu \| \mathbf{w}_1'', 256) \tag{3}$$

4. Let $\sigma' := (\tilde{c}', \mathbf{z}, \mathbf{h})$ and send the message m' with the purported signature σ' to the device for verification. Note that the components $\mathbf{z}$ and $\mathbf{h}$ are taken from the genuine signature σ.

5. During the signature verification procedure on the device, the attacker injects a fault to suppress subtracting $c\mathbf{t}_1$ in Algorithm 1, Algorithm 1.

If the fault injection is successful, the device will compute the same value $\mathbf{w}_1''$ as the attacker in Step 2 of the attack. Hence, the second part of the test in Algorithm 1, Algorithm 1 becomes a test that $\tilde{c}' = \texttt{SHAKE256}(\mu \| \mathbf{w}_1'', 256)$. This is of course correct. The other two parts of this test pass as well, because they are carried out on components $\mathbf{z}$ and $\mathbf{h}$ of a genuine signature. Therefore, the device will accept σ' as a valid signature for m'.

Of course the critical step is the fault injection in Step 5.

There is a variant of this attack. In this variant, the attacker changes step 3 in the preparation of the purported signature to:

$$\tilde{c}' := \texttt{SHAKE256}(\mu, 256) \tag{4}$$

In the fault injection step, the attacker skips the inclusion of $\mathbf{w}_1'$ in the `SHAKE256` computation. Note that with the usual absorb-and-squeeze API of `SHAKE256` this means a single instruction skip of a function call is sufficient to achieve this.

4 Falcon signature verification

We describe the signature verification procedure of Falcon. For a full description of the Falcon signature scheme we refer the reader to the specification of Falcon in [15].

There are two variants of Falcon, namely Falcon-512 and Falcon-1024. The two variants target NIST security levels 1 and 5, respectively.

Their mathematical descriptions differ only in the choice of some parameters. Most mathematical operations in Falcon signature verification are happening in a truncated polynomial ring $\mathbb{Z}_q[x]/(\phi)$. Here, $q = 12289$ (a prime number)

and $\phi(x) = x^n + 1$. We have $n = 512$ for FALCON-512 and $n = 1024$ for FALCON-1024. Another parameter is a rejection bound, which is $\lfloor \beta^2 \rfloor = 34\,034\,726$ for FALCON-512 and $\lfloor \beta^2 \rfloor = 70\,265\,242$ for FALCON-1024.

A message m in FALCON is simply a string of bytes. A FALCON signature is a pair (r, s), where r is a random 320-bit string and s is the representation of a polynomial $s_2 \in \mathbb{Z}_q[x]/(\phi)$. A FALCON public key is a polynomial $h \in \mathbb{Z}_q[x]/(\phi)$.

Furthermore, FALCON signature verification uses a function `HashToPoint` that maps a byte string to a polynomial in $c \in \mathbb{Z}_q[x]/(\phi)$ and a function `Decompress` that recovers the polynomial $s_2 \in \mathbb{Z}_q[x]/(\phi)$ from its representation as a string of bytes.

Algorithm 2 FALCON.VERIFY(m, `sig`, `pk`)

Require: A message m, a signature `sig` $= (r, s)$, a public key `pk` $= h \in \mathbb{Z}_q[x]/(\phi)$
Ensure: Accept or reject

1: $c \leftarrow$ `HashToPoint`$(r \| m)$
2: $s_2 \leftarrow$ `Decompress`(s)
3: **if** $s_2 = \perp$ **then**
4: reject
5: $s_1 \leftarrow c - s_2 h \bmod q$
6: **if** $\|(s_1, s_2)\|^2 \leq \lfloor \beta^2 \rfloor$ **then**
7: accept
8: **else**
9: reject

5 Fault attacks against Falcon signature verification

The most obvious target for a fault attack against FALCON signature verification is the comparison $\|(s_1, s_2)\| < \beta^2$, cf. Algorithm 2 in Algorithm 2. The goal of the attack is to inject a fault such that the comparison is skipped for an arbitrary (s_1, s_2).

However, because this is the most obvious place to attack, it is also a part of the code that a careful designer most likely implements with appropriate countermeasures against this type of attack. In this section, we are considering attacks which are less obvious and less likely to be prevented by countermeasures in the implementation.

We assume the attacker has a valid signature (r, s) for some message m. Every attack starts with the attacker choosing a message m'. All attacks aim at constructing a (forged) signature such that it only takes a single fault that can be injected with high probability to force the verifying device into accepting the signature and hence processing m'.

5.1 Fault injection during the inclusion of the message hash

For our first attack against FALCON signature verification, the attacker performs the following steps:

1. The attacker chooses r' to be an arbitrary string of 320 bits and sets $s' := \texttt{Compress}(0, n - 328)$, where $n = 512$ for FALCON-512 and $n = 1024$ for FALCON-1024. So s' is the compressed representation of the zero-polynomial.

2. The attacker sends the message m' and the purported signature (r', s') to the device for verification.

3. During the verification procedure, the attacker injects a fault to suppress the inclusion of c in Algorithm 2, Algorithm 2. So the device computes $s'_1 \leftarrow -s'_2 h \bmod q = 0$, because s'_2 is 0 by construction.

If the fault injection is successful, the check in Algorithm 2 becomes a check that $\lfloor \beta^2 \rfloor \geq \|(s'_1, s'_2)\| = 0$. This check obviously passes, and the device accepts the signature and starts processing m'.

We note that this attack does not work when FALCON is operated in key-recovery mode, see the FALCON-specification, Section 3.12 in [15], because Algorithm 2 in Algorithm 2, the line we attack, does not exist in the signature verification procedure in key-recovery mode.

5.2 Fault injection during the computation of the Euclidean norm

This fault injection attack targets the computation of the Euclidean norm in Algorithm 2, Algorithm 2.

Now let $i > 0$ be the smallest index such that $h_i \neq 0$[3].

So the public key h has the form:

$$h(x) = h_0 + \sum_{k=i}^{n-1} h_k x^k \tag{5}$$

We first explain how the attacker proceeds and then show why the attack works:

1. The attacker chooses a polynomial $s'_2 \in \mathbb{Z}_q/(\phi)$ with $s'_{2,i} = c_0 h^{-1}_{n-i}$ and $s_{2,j} = 0$ for $j \neq i$.

2. The attacker chooses a random 320-bit string r', sets $s' := \texttt{Compress}(s'_2, 8 * \texttt{sbytelen} - 328)$ and sends the chosen message m' together with the purported signature (r', s') to the device for processing.

[3]Note that such an i exists unless the public key h consists only of a constant term.

3. The attacker injects a fault such that the computation of the Euclidean norm in Algorithm 2, Algorithm 2 stops after processing the 0^{th} coefficient. We will show that this is a realistic attack target in Section 6.2.

To understand why this attack works (assuming, of course, that the fault injection is successful), we carefully follow the steps of the signature verification process. First, the device computes $c' \leftarrow \texttt{HashToPoint}(r' \| m', q, n)$ and decompresses s' to recover the s'_2 the attacker has constructed in the first step of the attack.

Next, the device computes $s'_1 \leftarrow c' - s'_2 h$. If we look at the 0^{th} coefficient of the polynomial s'_1, we see that

$$s'_{1,0} = c'_0 - s'_{2,i} h_{n-i} = c'_0 - c_0 h_{n-i}^{-1} h_{n-1} = 0, \tag{6}$$

because of the way the attacker has constructed s'_2.

If the fault injection in the last step of the attack is successful, then the device computes the Euclidean norm of (s'_1, s'_2) as

$$s'^2_{1,0} + s'^2_{2,0} = 0 + 0 = 0, \tag{7}$$

because the fault terminates the loop after the first iteration and other coefficients of s'_1 and s'_2 are ignored.

Hence, the norm is short enough for the device to accept the purported signature, and the device will process m' like a valid message.

Of course $s'_{1,0}$ does not have to be zero, it is sufficient if $s'^2_{1,0} \leq \lfloor \beta^2 \rfloor$. This is the case with some probability anyway.

So, instead of constructing s'_2 as above, an attacker can also choose $s'_2 = 0$ and then try different r' until $c'^2_0 \leq \lfloor \beta^2 \rfloor$, with $c' = \texttt{HashToPoint}(r' \| m')$. We see from Algorithm 2 in Algorithm 2 that then $s'^2_{1,0} = c'^2_0 \leq \lfloor \beta^2 \rfloor$. Because $\lfloor \beta^2 \rfloor$ is larger for Falcon-1024 than for Falcon-512, this simplified version of the attack is easier for Falcon-1024.

More precisely, we see that for Falcon-512 we have to have $c'^2_0 \leq 34\,034\,726$ and hence $0 \leq c'_0 \leq 5\,833$ and for Falcon-1024 we have to have $c'^2_0 \leq 70\,265\,242$ and hence $0 \leq c'_0 \leq 8\,382$. Note that $0 \leq c'_0 < q = 13329$ anyway.

6 Practical evaluation

To assess the practical validity of our attacks, we simulated fault injection on a STMF407G-DISC1 board featuring a 32-bit ARM Cortex-M4 (`STMF407VGT6`) high-performance microcontroller with FPU core, 1-Mbyte Flash memory, and 192-Kbyte RAM with the aid of a debugger (`GNU gdb 13.1`).

We ran our attacks against the `pqm4` library[8][4] compiled with `gcc version 12.2.0` and optimization flags `-O3 -mfloat-abi=hard -mfpu=fpv4-sp-d16`.

Note that such attacks would similarly work on the reference implementations of Dilithium [19] and Falcon [14] submitted at NIST.

[4] `commit a525417134995302bb5013dd112dec65cdb28ca9`

Listing 1: Excerpt from the DILITHIUM-2 signature verification function `crypto_sign_verify` in file `crypto_sign/dilithium2/m4f/sign.c`

```
274  ...
275  /* Matrix-vector multiplication; compute Az - c2^dt1 */
276  poly_challenge(&cp, c);
277  polyvec_matrix_expand(mat, rho);
278
279  polyvecl_ntt(&z);
280  polyvec_matrix_pointwise_montgomery(&w1, mat, &z);
281
282  poly_ntt(&cp);
283  polyveck_shiftl(&t1);
284  polyveck_ntt(&t1);
285  polyveck_pointwise_poly_montgomery(&t1, &cp, &t1);
286
287  polyveck_sub(&w1, &w1, &t1);
288  polyveck_reduce(&w1);
289  polyveck_invntt_tomont(&w1);
290  ...
```

6.1 Experiments with Dilithium

For the sake of evaluation, the implementation of DILITHIUM-2 was considered.

The fault attack described in Section 3 targets `polyveck_sub` operation is the function call in line 287 in file `crypto_sign/dilithium2/m4f/sign.c`. Skipping this function call has exactly the effect required in Step 5 of the attack (see listing 1).

This can be achieved on an ARM Cortex-M4 by skipping the branch with link (`bl`) instruction at address `0x80030c6`, which corresponds to the machine code location +254 in the `crypto_sign_verify` function located in the source file `crypto_sign/dilithium2/m4f/sign.c`, and jumps to the address `0x80028f8` corresponding to the `polyveck_sub` operation.

If the memory for `cp` is pre-initialized with zero, then skipping the call to `poly_challenge()` in Line 276 in listing 1 has a similar effect. We did not pursue this variant of the attack experimentally.

Note that a seminal idea of removing the term $c\mathbf{t}_1$ from the computation in Algorithm 1, Algorithm 1 can also be found in [17]. However, in [17] the target of the attack is the number-theoretic transform, which requires a slightly more involved construction of the forged signature and a fault that zeroes data or changes a pointer to point into an array filled with zeroes. As skipping instructions is a well-documented effect of fault injections, we believe our attack to be easier in practice.

Also, a similar idea, this time in the context of the signature scheme ring-TESLA, can be found in [2]. Again, the fault model used in [2] is zeroing of

Listing 2: Assembly code corresponding to the DILITHIUM-2 signature verification function `crypto_sign_verify` in the source code file `crypto_sign/dilithium2/m4f/sign.c`

```
1  ...
2  0x80030c4 <crypto_sign_verify+252> adds r2, #16
3  0x80030c6 <crypto_sign_verify+254> bl 0x80028f8 <polyveck_sub>
4  0x80030ca <crypto_sign_verify+258> add.w r0, sp, #10304
5  ...
```

Listing 3: Excerpt from the DILITHIUM-2 signature verification function `crypto_sign_verify` in file `crypto_sign/dilithium2/m4f/sign.c`

```
295      ...
296      /* Call random oracle and verify challenge */
297      shake256_inc_init(&state);
298      shake256_inc_absorb(&state, mu, CRHBYTES);
299      shake256_inc_absorb(&state, buf, K*
             POLYW1_PACKEDBYTES);
300      shake256_inc_finalize(&state);
301      shake256_inc_squeeze(c2, SEEDBYTES, &state);
```

data.

Let us now look at the second attack against DILITHIUM signature verification described in Section 3. Exemplary source code for the targeted code section is shown in listing 3. The attacker has to inject a fault such that the function call in line 299 is skipped. As we have just described skipping a function call with a fault injection, we are not going to repeat details here.

6.2 Experiments with Falcon

For the sake of evaluation, the implementation of FALCON-512 was considered.

The first attack described in Section 5.1 requires skipping the inclusion of the message hash c.

listing 4 shows an excerpt of the `pqm4` implementation of the Falcon signature verification procedure. The excerpt corresponds to Algorithm 2 in Algorithm 2. The buffer `tt` contains the polynomial s_2.

The targeted operation is the function call in line 671 in the source file `crypto_sign/falcon-512/m4-ct/vrfy.c`. If the attacker manages to skip the function call in this line, the attack is successful for the reasons explained in Section 5.1.

This could be achieved on an ARM Cortex-M4 by skipping the branch with link (`bl`) instruction to the function `mq_poly_sub`.

However, the ARM Cortex-M4 machine code translation does not generate

Listing 4: Excerpt from the FALCON-512 signature verification function `verify_raw` in file `crypto_sign/falcon-512/m4-ct/vrfy.c`

```
664  ...
665  /*
666   * Compute -s1 = s2*h - c0 mod phi mod q (in tt[]).
667   */
668  mq_NTT(tt, logn);
669  mq_poly_montymul_ntt(tt, h, logn);
670  mq_iNTT(tt, logn);
671  mq_poly_sub(tt, c0, logn);
672
673  /*
674   * Normalize -s1 elements into the [-q/2..q/2] range.
675   */
676  for (u = 0; u < n; u ++) {
677          int32_t w;
678
679          w = (int32_t)tt[u];
680          w -= (int32_t)(Q & -(((Q >> 1) - (uint32_t)w)
                 >> 31));
681          ((int16_t *)tt)[u] = (int16_t)w;
682  }
683
684  /*
685   * Signature is valid if and only if the aggregate (-s1
         ,s2) vector
686   * is short enough.
687   */
688  return Zf(is_short)((int16_t *)tt, s2, logn);
689  ...
```

Table 2: FALCON-512: Success rate of instruction skip on `bne.n` instruction over number of coefficients of c. The numbers are the result of 50 experiments per column with the `pqm4` library.

# of coefficients	1	2	3	4	5	6
Success rate	1	0.81	0.61	0.32	0.19	0.08

a function call to `mq_poly_sub` when compiled with the option `-O3`, because the function gets inlined by the compiler, being defined and used only once in the same file `crypto_sign/falcon-512/m4-ct/vrfy.c`. listing 5 shows the corresponding assembly code. Note that the instructions on lines 4-8 correspond to the function `mq_sub`, which also gets inlined by the compiler, cf. line 629 in listing 6.

Listing 5: Assembly code corresponding to the FALCON-512 signature verification function `verify_raw` in file `crypto_sign/falcon-512/m4-ct/vrfy.c`

```
1  ...
2  0x800d834 <verify_raw+136> bl     0x800d62c <mq_iNTT>
3  0x800d83a <verify_raw+142> movw   r1, #12289
4  0x800d83e <verify_raw+146> ldrh.w r3, [r4, #2]!
5  0x800d842 <verify_raw+150> ldrh.w r2, [r6, #2]!
6  0x800d846 <verify_raw+154> subs   r3, r3, r2
7  0x800d848 <verify_raw+156> and.w  r2, r1, r3, asr #31
8  0x800d84c <verify_raw+160> add    r3, r2
9  0x800d84e <verify_raw+162> cmp    r9, r4
10 0x800d850 <verify_raw+164> strh   r3, [r4, #0]
11 0x800d852 <verify_raw+166> bne.n  0x800d83e <verify_raw+146>
12 ...
```

In this case, one option is to choose r such that the first coefficient of c is small enough, e.g., choose r such that the first coefficient of c is zero, and perform an instruction skip of `bne.n` to early-abort the loop in the `mq_poly_sub` function call (cf. listing 6) and pass the final comparison. Note that the attack works even if multiple instructions around the `bne.n` are skipped, e.g., all instructions from `verify_raw+146` to `verify_raw+166` can be safely skipped, and the attack would still be successful. Furthermore, the attack works even if multiple coefficients are considered, i.e., the `bne.n` instruction is skipped after a certain number of iterations. The success probability for randomly chosen r and m' over the number of coefficients considered is shown in Table 2. It can be observed that the success probability decreases as multiple coefficients of c are included in the norm computations. This shows that there are multiple ways of attacking the `mq_poly_sub` and the attack is still effective in the considered attack scenario.

A second option is to set `logn` to zero in order to early-abort the for loop, cf. line 627 in listing 6.

The second fault attack, described in Section 5.2, targets the computation of the Euclidean norm in Algorithm 2, Algorithm 2.

There are a number of ways to attack this. This is best shown by looking at the `pqm4` source code. It seems plausible that other implementations are similar

Listing 6: Excerpt from the FALCON-512 signature verification function `my_poly_sub` in file `crypto_sign/falcon-512/m4-ct/vrfy.c`

```
618  ...
619  /*
620   * Subtract polynomial g from polynomial f.
621   */
622  static void
623  mq_poly_sub(uint16_t *f, const uint16_t *g, unsigned
         logn)
624  {
625          size_t u, n;
626
627          n = (size_t)1 << logn;
628          for (u = 0; u < n; u ++) {
629                  f[u] = (uint16_t)mq_sub(f[u], g[u]);
630          }
631  }
632  ...
```

because there is an obvious way to compute the Euclidean norm of a vector.

The verification function `verify_raw()` in [14] calls a function `is_short()` in line 688[5].

This function verifies that the signature (s_1, s_2) is short.

The function `is_short()` is implemented in `common.c` and shown in listing 7.

One possible target for a fault injection is the passing of the parameter `logn` when `is_short()` is called.

An attacker may be able to force `logn` to zero. In this case, the loop in lines 252–261 that computes the Euclidean norm stops after the first iteration.

Alternatively, the attacker could inject a fault to skip the jump-instruction at the end of the loop body in line 261 back to the beginning of the loop. Again, this would lead to the loop terminating after the first iteration.

This can be observed by looking at the assembly code of listing 8. In this case is sufficient to skip the shift instruction `lsls` or by zeroing the content of register `r2` or `r5` before executing it.

Listing 8: Assembly code corresponding to the FALCON-512 signature verification function `is_short` in file `crypto_sign/falcon-512/m4-ct/common.c`

```
1  ...
2  0x8000d7a <is_short+2>  movs    r5, #2
3  0x8000d7c <is_short+4>  subs    r0, #2
4  0x8000d7e <is_short+6>  lsls    r5, r2
5  ...
6  0x8000d9c <is_short+36> cmp     r5, r0
```

[5]Note that `Zf` is a macro that just puts a pre-fix in front of the function name; we can safely ignore this for our purposes here.

Listing 7: Excerpt from the FALCON-512 auxiliary function that computes the Euclidean length of a vector in file `crypto_sign/falcon-512/m4-ct/common.c`.

```
237  int
238  Zf(is_short)(
239          const int16_t *s1, const int16_t *s2, unsigned
                 logn)
240  {
241      /*
242       * We use the l2-norm. Code below uses only 32-bit
             operations to
243       * compute the square of the norm with saturation to
              2^32-1 if
244       * the value exceeds 2^31-1.
245       */
246      size_t n, u;
247      uint32_t s, ng;
248
249      n = (size_t)1 << logn;
250      s = 0;
251      ng = 0;
252      for (u = 0; u < n; u ++) {
253          int32_t z;
254
255          z = s1[u];
256          s += (uint32_t)(z * z);
257          ng |= s;
258          z = s2[u];
259          s += (uint32_t)(z * z);
260          ng |= s;
261      }
262      s |= -(ng >> 31);
263
264      /*
265       * Acceptance bound on the l2-norm is:
266       *   1.2*1.55*sqrt(q)*sqrt(2*N)
267       * Value 7085 is floor((1.2^2)*(1.55^2)*2*1024).
268       */
269      return s < (((uint32_t)7085 * (uint32_t)12289) >>
             (10 - logn));
270  }
```

Listing 9: Assembly code corresponding to the FALCON-512 signature verification function `is_short` in file `crypto_sign/falcon-512/m4-ct/common.c`

```
 1  ...
 2  0x8000da4 <is_short+44> ldr     r0, [pc, #20]
 3  0x8000da6 <is_short+46> rsb     r2, r2, #10
 4  0x8000daa <is_short+50> lsrs    r0, r2
 5  0x8000dac <is_short+52> orr.w   r12, r12, r4, asr #31
 6  0x8000db0 <is_short+56> cmp     r0, r12
 7  0x8000db2 <is_short+58> ite     ls
 8  0x8000db4 <is_short+60> movls   r0, #
 9  0x8000db6 <is_short+62> movhi   r0, #1
10  ...
```

```
 7  0x8000da2 <is_short+42> bne.n   0x8000d88 <_is_short+16>
 8  ...
```

Note that the comparison in line 269 in listing 7 will pass, even if the attacker forces `logn` to zero, because all values are unsigned integers.

This can be observed by looking at the assembly code of listing 9.

7 Countermeasures

Obviously, generic countermeasures against manipulation of the control flow, e.g., hardware instruction skip, make most of the attacks presented in this paper more difficult to realize in practice.

However, there are also more specific countermeasures that can be implemented to thwart our attacks.

For instance, against the attack described in Section 3, one can proceed as follows: the implementation generates a random $\mathbf{u} \in R_q^k$ and then computes $\mathbf{Az} + \mathbf{u}$ and $c\mathbf{t}_1 + \mathbf{u}$. The attacked step in Algorithm 1, Algorithm 1 then becomes

$$(\mathbf{Az} + \mathbf{u}) - (c\mathbf{t}_1 + \mathbf{u}). \tag{8}$$

If the attacker skips this subtraction, then, with overwhelming probability, the value $\mathbf{w}_1'$ computed in Algorithm 1, Algorithm 1 is not the value $\mathbf{w}_1''$ calculated by the attacker. Hence, signature verification will most likely fail.

A similar countermeasure can be used against the attack in Section 5.1. We modify the implementation of Algorithm 2 to generate a random $u \in \mathbb{Z}_q[x]/(\phi)$.

Then u is added to c and to s_2h. Consequently, Algorithm 2 in Algorithm 2 becomes

$$s_1 \leftarrow (c + u) - (s_2h + u) \bmod q. \tag{9}$$

If the attacker skips the inclusion of $c + u$ in this operation, then the test in Algorithm 2 will fail with very high probability.

Finally, by checking the integrity of function parameters and introducing redundant loop counters seem to be efficient countermeasures against the attack in Section 5.2.

8 Conclusion and future work

We have shown several fault injection attacks against non-obvious targets in the verification routines of Dilithium and Falcon which allow obtaining forged signatures accepted for arbitrary messages.

The attacks aim to demonstrate potential vulnerabilities in implementations of the verification routines of Dilithium and Falcon which are non-obvious targets for fault attacks and which can hence be expected to have been left unprotected even by experienced designers.

Following up from this work, it would be interesting to transfer our attacks to other lattice-based signature schemes. Applying our attacks against Falcon to the signature verification procedure of Mitaka [6], for example, seems straightforward. Generalizing the attack techniques in this paper to target also ModFalcon [4] would similarly be of interest.

Another line of research would be to consider further implementations of Dilithium and Falcon. At the moment, however, most implementations of the signature verification procedure seem to be based on the reference code that was part of the respective submission to the NIST standardization project.

References

[1] Sven Bauer and Fabrizio De Santis. *A Differential Fault Attack against Deterministic Falcon Signatures*. Cryptology ePrint Archive, Paper 2023/422. `https://eprint.iacr.org/2023/422`. 2023. URL: `https://eprint.iacr.org/2023/422`.

[2] Nina Bindel, Johannes Buchmann, and Juliane Krämer. *Lattice-Based Signature Schemes and their Sensitivity to Fault Attacks*. Cryptology ePrint Archive, Report 2016/415. `https://eprint.iacr.org/2016/415`. 2016.

[3] Leon Groot Bruinderink and Peter Pessl. "Differential Fault Attacks on Deterministic Lattice Signatures". In: *IACR TCHES* 2018.3 (2018). `https://tches.iacr.org/index.php/TCHES/article/view/7267`, pp. 21–43. ISSN: 2569-2925. DOI: `10.13154/tches.v2018.i3.21-43`.

[4] Chitchanok Chuengsatiansup et al. "ModFalcon: Compact Signatures Based On Module-NTRU Lattices". In: *ASIACCS 20*. Ed. by Hung-Min Sun et al. ACM Press, Oct. 2020, pp. 853–866. DOI: `10.1145/3320269.3384758`.

[5] Jean-Max Dutertre et al. *Experimental Analysis of the Laser-induced Instruction Skip Fault Model*. Accessed 2023-05-03. URL: `https://hal.science/hal-02379754/document`.

[6] Thomas Espitau et al. *Mitaka: a simpler, parallelizable, maskable variant of Falcon*. Cryptology ePrint Archive, Report 2021/1486. https://eprint.iacr.org/2021/1486. 2021.

[7] Andreas Hülsing et al. *SPHINCS$^+$*. Tech. rep. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022. National Institute of Standards and Technology, 2022.

[8] Matthias J. Kannwischer et al. *pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4*. Cryptology ePrint Archive, Report 2019/844. https://eprint.iacr.org/2019/844. 2019.

[9] Vadim Lyubashevsky et al. *CRYSTALS-DILITHIUM*. Tech. rep. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022. National Institute of Standards and Technology, 2022.

[10] Sarah McCarthy et al. *BEARZ Attack FALCON: Implementation Attacks with Countermeasures on the FALCON signature scheme*. Cryptology ePrint Archive, Report 2019/478. https://eprint.iacr.org/2019/478. 2019.

[11] Alexandre Menu et al. *Experimental Analysis of the Electromagnetic Instruction Skip Fault Model*. Accessed 2023-05-03. URL: https://hal.science/hal-02572398/document.

[12] Nicolas Moro et al. *Formal verification of a software countermeasure against instruction skip attacks*. Cryptology ePrint Archive, Report 2013/679. https://eprint.iacr.org/2013/679. 2013.

[13] James A. Muir. "Seifert's RSA Fault Attack: Simplified Analysis and Generalizations". In: *ICICS 06*. Ed. by Peng Ning, Sihan Qing, and Ninghui Li. Vol. 4307. LNCS. Springer, Heidelberg, Dec. 2006, pp. 420–434.

[14] Thomas Pornin. *Falcon source files (reference implementation) vrfy.c*. Accessed 2023-05-03. URL: https://falcon-sign.info/impl/vrfy.c.html.

[15] Thomas Prest et al. *FALCON*. Tech. rep. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022. National Institute of Standards and Technology, 2022.

[16] Prasanna Ravi, Anupam Chattopadhyay, and Anubhab Baksi. *Side-channel and Fault-injection attacks over Lattice-based Post-quantum Schemes (Kyber, Dilithium): Survey and New Results*. Cryptology ePrint Archive, Report 2022/737. https://eprint.iacr.org/2022/737. 2022.

[17] Prasanna Ravi et al. "Fiddling the Twiddle Constants - Fault Injection Analysis of the Number Theoretic Transform". In: *IACR TCHES* 2023.2 (2023), pp. 447–481. DOI: 10.46586/tches.v2023.i2.447-481.

[18] Jean-Pierre Seifert. "On Authenticated Computing and RSA-Based Authentication". In: *Proceedings of the 12th ACM Conference on Computer and Communications Security*. CCS '05. Alexandria, VA, USA: Association for Computing Machinery, 2005, pp. 122–127. ISBN: 1595932267. DOI: 10.1145/1102120.1102138. URL: https://doi.org/10.1145/1102120.1102138.

[19] Gregor Seiler et al. *Dilithium reference implementation V3.1 on github*. Accessed 2023-04-28. URL: https://github.com/pq-crystals/dilithium/tree/v3.1.

[20] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. Tech. rep. Federal Information Processing Standards Publications (FIPS PUBS) 204 August 13, 2024. Washington, D.C.: U.S. Department of Commerce, 2001. DOI: 10.6028/NIST.FIPS.204. URL: https://doi.org/10.6028/NIST.FIPS.204.

[21] National Institute of Standards and Technology. *NIST Announces First Four Quantum-Resistant Cryptographic Algorithms*. https://www.nist.gov/news-events/news/2022/07/nist-announces-first-four-quantum-resistant-cryptographic-algorithms. Accessed 2022-12-21. 2022.

[22] National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard*. Tech. rep. Federal Information Processing Standards Publications (FIPS PUBS) 205 August 13, 2024. Washington, D.C.: U.S. Department of Commerce, 2001. DOI: 10.6028/NIST.FIPS.205. URL: https://doi.org/10.6028/NIST.FIPS.205.

[23] National Institute of Standards and Technology. *Submission Requirements and Evaluation Criteria for the Post-Quantum Cryptography Standardization Process*. accessed 2023-05-09. 2016. URL: https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf.